

Лабораторная работа №2. Модель нейрона

1. Физиологическая модель нейрона

Нейрон (нервная клетка) состоит из:

- тела клетки (cell body),
- внешних ветвей:
 - аксона (axon)
 - дендритов (dendrites).

Тело клетки включает:

- ядро (nucleus), которое содержит информацию о наследственных свойствах,
- плазму, обладающую молекулярными средствами для производства необходимых нейрону материалов.

Нейрон получает сигналы (импульсы) от других нейронов через дендриты (приемники) и передает сигналы, сгенерированные телом клетки, вдоль аксона (передатчик), который в конце разветвляется на волокна (strands). На окончаниях этих волокон находятся синапсы (synapses).

Практические задания для самостоятельной работы:

Задание 1:

1. Постройте графики активационных функций.
2. Постройте графики производных активационных функций.
3. Сделайте сравнительный анализ.

Задание 2:

1. Постройте 2 пространства из 10 объектов из 2-х классов (по 5 примеров класса, 2 координаты) в каждом:
 - а) пространство линейно делимых объектов,
 - б) пространство линейно не делимых объектов.
2. Постройте нейрон и проведите его обучение на 2 пространствах по Дельта-правилу.
3. Запишите траекторию изменения значений синаптических параметров и визуализируйте их.
4. Оцените характер изменения значений синаптических весов.

Вспомогательный материал

Задание 1. Создать формальную модель нейрона

1.1. Нейрон МакКалока-Питтса

Пример

```
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
[:
def McCulloh_Pitts_neuron(x = None, w = None, bias = 0):
    # w - веса
    # x - входы
    #print('X: ',x)
    #print('W: ',w)
    #print('bias: ',bias)
    s = np.dot(w, x.T) + bias
    #print('суммарный уровень входов нейрона на их веса: ',s)
    y = np.sign(s)
    return y
```

Запустим нейрон на примере x с весами w

```
x = np.array([1,0])
w = np.array([1,-1])

y_out = McCulloh_Pitts_neuron(x = x, w = w, bias = 0.3)
print('ответ нейрона',y_out)
```

ответ нейрона 1.0

```
x = np.random.random((10,2))
w = np.array([1,-1])
y_out1 = McCulloh_Pitts_neuron(x = x, w = w, bias = 0.3)
print('ответ нейрона',y_out1)
```

ответ нейрона [1. 1. -1. -1. 1. -1. 1. -1. -1. -1.]

1.2. Линейный нейрон

$y = XW + b$

```
def linear_neuron(x = None, w = None, bias = 0):
    #print('X: ',x)
    #print('W: ',w)
    #print('bias: ',bias)
    s = np.dot(w, x.T) + bias
    #print('суммарный уровень входов нейрона на их веса: ',s)
    y = s
    return y
```

Запустим нейрон на множестве примеров в x

```
x = np.random.random((10,2))
w = np.array([1,-1])
y_out1 = linear_neuron(x = x, w = w, bias = 0.3)
print('ответ нейрона',y_out1)
```

```
ответ нейрона [-0.05584629 -0.09410133  0.12127347  0.71637982  0.55193246  0.29453727
 1.13532146  0.66715623  0.03077807 -0.03710919]
```

1.3. Сигмоидный нейрон

$s = X W + b, y = 1/(1 + \exp(-s))$

```
def sigmoid_neuron(x = None, w = None, bias=0):
    #print('X: ',x)
    #print('W: ',w)
    #print('bias: ',bias)
    s = np.dot(w, x.T) + bias
    #print('суммарный уровень входов нейрона на их веса: ',s)
    y = 1 / (1 + np.exp(-s))
    return y
```

Тангенциальный

$s = X W + b, y = 2/(1 + \exp(-s))-1$

```
def sigmoidt_neuron(x = None, w = None, bias=0):
    #print('X: ',x)
    #print('W: ',w)
    #print('bias: ',bias)
    s = np.dot(w, x.T) + bias
    #print('суммарный уровень входов нейрона на их веса: ',s)
    y = 2 / (1 + np.exp(-s))-1
    return y
```

Запустим нейрон

```
y_out2 = sigmoid_neuron(x = x, w = w, bias = 0.3)
print('ответ нейрона',y_out2)
```

Запустим все нейроны на множестве примеров (линейно равномерно распределены в интервале от -10 до 10)

```
x = np.linspace(-10,10.1,20).reshape((20,1))
w = np.array([0.9]).reshape((1,1))
bias = 0.3
```

```
y_out1 = McCulloh_Pitts_neuron(x = x, w = w, bias = 0.3)
y_out2 = sigmoid_neuron(x = x, w = w, bias = 0.3)
y_out3 = sigmoidt_neuron(x = x, w = w, bias = 0.3)
y_out4 = linear_neuron(x = x, w = w, bias = 0.3)
```

```
plt.plot(x[:,0].T,y_out1[0,:],'r', label='McCulloh-Pitts')
plt.plot(x[:,0].T,y_out2[0,:],'b', label='sigmoid')
plt.plot(x[:,0].T,y_out3[0,:],'g', label='sigmoid tanh')
#plt.plot(x[7:12,0].T,y_out4[0,7:12],'k', label='linear')
plt.legend()
plt.title('Ответы нейронов на x')
plt.xlabel('x')
plt.show()
```

Ответы нейронов для разных нейронов

Пороговый:

y_out1

Сигмоидный нейрон:

y_out2

Сигмоидный тангенциальный нейрон:

y_out3

Усложним сигмоидный нейрон

```
def sigmoidt_complex_neuron(x = None, w = None, bias=0, lymbd = 1):
```

```
    #print('X: ',x)
```

```
    #print('W: ',w)
```

```
    #print('bias: ',bias)
```

```
    s = np.dot(w, x.T) + bias
```

```
    #print('суммарный уровень входов нейрона на их веса: ',s)
```

```
    y = 2 / (1 + np.exp(-s/lymbd))-1
```

```
    return y
```

Посмотрим на работу нейрона

```
x = np.linspace(-10,10,20).reshape((20,1))
```

```
w = np.array([0.9]).reshape((1,1))
```

```
bias =0.3
```

```
for i in range(10):
```

```
    y_out3 = sigmoidt_complex_neuron(x = x, w = w, bias = 0.3, lymbd=0.1+i*0.5)
```

```
    plt.plot(x[:,0].T,y_out3[0,:], label='sigmoid tanh')
```

```
#plt.legend()
```

```
plt.title('Ответы нейронов на x')
```

```
plt.xlabel('x')
```

```
plt.show()
```

Задание 2. Создание модели двухмерного нейрона

Пример

Строим поле координат 10x10

```
x1 = np.array([np.linspace(-10,11,10).reshape((10,1))*10].reshape((10,10))
```

```
x2 = x1.T
```

```
x2=x2.reshape((100,1))
```

```
x1=x1.reshape((100,1))
```

```
print( x1[:3])
```

```
print(x2[:3])
```

```
x = np.hstack([x1,x2])
```

Размерность данных

```
x.shape
```

Строим поверхность двухмерного нейрона (чем желтее, тем ближе к 1, чем синее, тем ближе к -1)

```
w = np.array([0.9, -0.9]).reshape((1,2))
```

```
y_out_21 = McCulloh_Pitts_neuron(x = x, w = w, bias = 10)
```

```
y_out_21 = y_out_21.reshape((10,10))
```

```
plt.imshow((y_out_21 ))  
plt.show()
```

```
w = np.array([-0.9, 0.9]).reshape((1,2))
```

```
y_out_22 = McCulloh_Pitts_neuron(x = x, w = w, bias = 10)
```

```
y_out_22 = y_out_22.reshape((10,10))
```

```
plt.imshow((y_out_22 ))  
plt.show()
```

Сигмоидный нейрон на том же пространстве

```
# веса нейронов
```

```
w = np.array([0.9, -0.1]).reshape((1,2))
```

```
# выход нейрона
```

```
y_out_2s = sigmoidt_complex_neuron(x = x, w = w, bias = 33.9, lymbd=0.1+i*0.5)
```

```
y_out_2s = y_out_2s.reshape((10,10))
```

```
plt.imshow((y_out_2s +1)/2)  
plt.show()
```

```
w = np.array([-0.1, 0.1]).reshape((1,2)) # веса нейрона
```

```
# выход нейрона
```

```
y_out_21s = sigmoidt_complex_neuron(x = x, w = w, bias = 33.9, lymbd=0.1+i*0.5)
```

```
y_out_21s = y_out_21s.reshape((10,10))
```

```
plt.imshow((y_out_21s +1)/2)  
plt.show()
```

Все нейроны отличаются весами, что приводит к разным выходам.

Задание 3. Обучение нейрона по Дельта-правилу

2. Задать начальное состояние
3. Взять пример j из набора примеров
4. Вычислить выход нейрона для этого примера
5. Вычислить коррекцию синаптических весов для всех входов x_i

$dw_i = - \eta (y(X_j) - d_j) x_i$

5. Вычислить новый уровень w_i $w_i = w_i + dw_i$
6. Перейти к новому примеру в п.2
7. Если просмотрели все примеры, то проверить условие конца (ошибка модели или число повторов):
 - условие выполнено - конец

- условие не выполнено - начинаем просматривать примеры и изменять веса сначала (j=0) и к п.2

Эпоха - просмотр всех примеров из набора

Пример

```
# входное пространство
x1 = np.array([1,1,-1,-1])
x2 = np.array([1,-1,-1,1])

x = np.vstack([x1,x2]).T

# целевые значения для входов
D=np.array([1,1,-1,-1])
# рисуем
plt.plot(x1[:2],x2[:2], 'or', label = 'класс 1')
plt.plot(x1[2:4],x2[2:4], 'og', label = 'класс -1')
plt.legend()
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('Примеры объектов ')
plt.show()

plt.plot(D, 'b')
plt.title('Примеры значений класса ')
plt.title('целевой выход')
plt.xlabel('номер объекта')
plt.ylabel('D')
plt.show()
```

Посмотрим на входы

x

[28]:

```
array([[ 1,  1],
       [ 1, -1],
       [-1, -1],
       [-1,  1]])
```

Обучение нейрона

[]:

```
# Начальные веса
w=np.array([0.1,-0.1])

#yt = sigmoid_neuron(x = x, w = w, bias = 0)
yt = linear_neuron(x = x, w = w, bias = 0)
plt.plot(yt,'r' , label = 'выходы до обучения')

for k in range(3):
    for i in range(4):

        e = yt-D
        dw = - 0.1 * e[i] * x[i,:]
        w = w + dw
        #yt = sigmoid_neuron(x = x, w = w, bias = 0)
        yt = linear_neuron(x = x, w = w, bias = 0)
        plt.plot(yt,'--b')

plt.plot(yt,'--b', label = 'выходы на этапе обучения')
plt.plot(yt,'g', label = 'выходы после обучения')
plt.legend()
```

```
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

На графике видно, как изменяется выход нейрона при обучении (постепенно сдвигаем к целевому значению)

Задание 4. Линейная разделимость

Пример. Создание линейно-разделимой модели

[]:

```
# зададим примеры на вход
x1 = np.array([1,1,-1,-1])
x2 = np.array([1,-1,-1,1])

x = np.vstack([x1,x2]).T
# зададим примеры на выход
D=np.array([1,-1,1,-1])

plt.plot(x1[[1,3]],x2[[1,3]], 'or', label = 'класс 1')
plt.plot(x1[[0,2]],x2[[0,2]], 'og', label = 'класс -1')
plt.legend()
plt.xlabel('x1')
plt.ylabel('x2')
plt.title('Примеры объектов ')
plt.show()

plt.plot(D, 'b')
plt.title('Примеры значений класса ')
plt.title('целевой выход')
plt.xlabel('номер объекта')
plt.ylabel('D')
plt.show()

# Начальные веса
w=np.array([0.1,-0.1])

#yt = sigmoid_neuron(x = x, w = w, bias = 0)
yt = linear_neuron(x = x, w = w, bias = 0)

plt.plot(yt,'r' , label = 'выходы до обучения')

# Обучение
for k in range(3):
    for i in range(4):

        e = yt-D
        dw = - 0.1 * e[i] * x[i,:]
        w = w + dw
        #yt = sigmoid_neuron(x = x, w = w, bias = 0)
        yt = linear_neuron(x = x, w = w, bias = 0)
        plt.plot(yt,'--b')

plt.plot(yt,'--b', label = 'выходы на этапе обучения')
plt.plot(yt,'g', label = 'выходы после обучения')
plt.legend()
plt.xlabel('x')
plt.ylabel('y')
plt.show()
```

Объединим ответы 2-х нейронов в третьем (выходы 2-х первых - входы третьего)

```
x_y2=np.hstack([y_out_21.reshape((100,1)),y_out_22.reshape((100,1))])  
x_y2.shape
```

[]:

[111]:

(100, 2)

Запустим нейрон и оказывается , что он может объединить решения нейронов , т.е. 1 выставаем только там, где оба входных нейрона выставили 1. Остальные области помечены 0.

Литература

1. Уоссермен Ф. Нейрокомпьютерная техника: Теория и практика. — М.: «Мир», 1992. Архивная копия от 30 июня 2009 на Wayback Machine
2. Хайкин С. Нейронные сети: Полный курс. Пер. с англ. Н. Н. Куссуль, А. Ю. Шелестова. 2-е изд., испр. — М.: Издательский дом Вильямс, 2008, 1103 с.